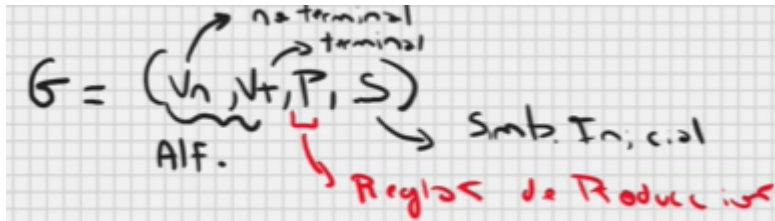


Clase 2

Gramáticas formales

Las gramáticas por lo general se denotan la letra G y, a nivel matemático, son un conjunto integrado por V_n , V_t , P , S .



En la clase anterior definimos el lenguaje por completitud, sabiendo los elementos y combinándolos.

V_n y V_t son finitos, y V_n y V_t no comparten elementos.

La regla de producción es un par ordenado:

Handwritten equation $P = (V_n^* - V_t^*) \times V^*$. Brackets below indicate (a) for $(V_n^* - V_t^*)$ and (b) for V^* .

Sin embargo, esa es la notación matemática, mientras que lo que solemos usar es:

Handwritten equation $a \rightarrow b$.

V_t (vocabulario terminal) son los elementos terminales del lenguaje, caracteres, letras, que no derivan en nada más.

V_n (vocabulario no terminal) son las variables o generadores del lenguaje. Suelen ser mayúsculas.

El elemento inicial de las reglas de producción (generalmente S) debe pertenecer a los elementos no terminales.

Ejemplo:

Handwritten example of a grammar $G = (\underbrace{\{S, X, Y\}}_{V_n}; \underbrace{\{0, 1\}}_{V_t}, P, S)$.

Reglas de producción:

$$P \begin{cases} \underline{S \rightarrow 0S} & \underline{S \rightarrow 10X} \\ \underline{X \rightarrow 1X} & \underline{1X \rightarrow 0Y} \\ \underline{Y \rightarrow 0} & \end{cases}$$

Derivaciones

$$\begin{aligned} S &\rightarrow 0S \rightarrow 00S \rightarrow \\ &\rightarrow 0010X \rightarrow 00101X \\ &\rightarrow 001010Y \rightarrow \boxed{0010100} \end{aligned}$$

$$\begin{aligned} S &\rightarrow 10X \rightarrow 100X \\ &\rightarrow \boxed{1000} \end{aligned}$$

Esta última es la palabra más corta.

Definiendo algunas reglas de producción y tomando decisiones en base a cuál derivación tomar podemos formar palabras dentro de un alfabeto. Ahora no sabemos las palabras, pero sí las reglas para generar palabras.

También podemos identificar una expresión que permita formar todas las palabras del lenguaje. Para eso, comenzamos analizando los caminos desde el símbolo inicial.

Como la primera regla es $S \rightarrow 0S$:

$$n \geq 0$$

$$0^n$$

El 0 va a estar por lo menos una vez si se toma esa derivación, pero si se comienza por la de $S \rightarrow 10X$ no. Las reglas de este estilo se llaman recurrencia.

Cuando hacemos $S \rightarrow 10X$ en vez de $S \rightarrow 0S$ nos queda:

$$n \geq 0$$

$$0^n 10$$

Sin embargo, analizando lo que sigue (las derivaciones de X), notamos que ese 10 no se va a volver a repetir, solo va a aparecer una vez.

Tomando la derivación $X \rightarrow 1X$ nos queda:

$$n \geq 0 \quad m \geq 0$$

$$0^n 1 0^m$$

Donde el 1 puede no estar nunca (por eso $m \geq 0$) ya que se podría tomar la otra regla de derivación siempre ($X \rightarrow 0Y$).

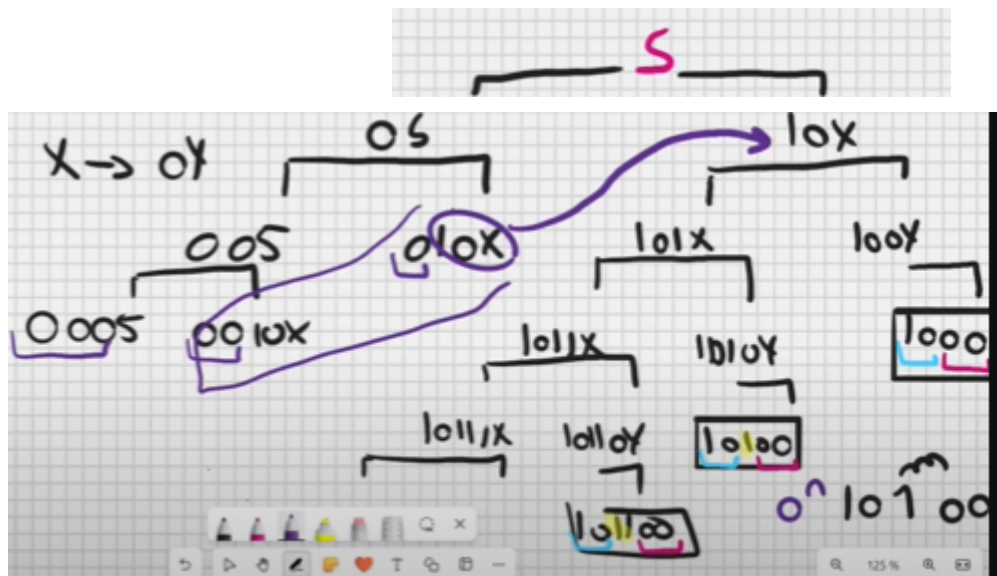
Al llegar a $X \rightarrow 0Y$, se nos generará como mucho un $0Y$, y al derivar Y como $Y \rightarrow 0$ nos queda:

$$n \geq 0 \quad m \geq 0$$

$$0^n 1 0^m 0 0$$

Forma genérica de las palabras.

Para no tener que expresar estas combinaciones de forma tan matemática, con exponentes y eso, se utilizan árboles de decisión:



Las hojas de este árbol son las palabras que podemos formar.

Otro ejemplo

$$G = (\{S, X, Y\}, \{a, b, c\}, P, S)$$

$$\begin{cases} S \rightarrow a S b & S \rightarrow a X \\ X \rightarrow c X & X \rightarrow b Y \\ Y \rightarrow a \end{cases}$$

¿Cuál es la palabra de menor longitud y cuál es su longitud? → ejercicio común, junto a la más larga.

Handwritten derivation on grid paper: $S \rightarrow aX \rightarrow aBy$, with a box around aba below it.

Cada gramática genera un único lenguaje, pero los lenguajes pueden ser generados por muchas gramáticas. Entonces, se pregunta cuál gramática es mejor para generar un lenguaje.

Se denominan gramáticas equivalentes a las cuales ante los mismos símbolos terminales pero con diferentes reglas de producción generan las mismas palabras.

Jerarquías gramaticales de Chomsky

Plantea 4 tipos:

- Gramáticas de tipo 0 → sin restricciones. Puede tener cualquier forma, es decir, cualquier gramática es de este tipo.
- Gramáticas de tipo 1 → sensibles al contexto. Son aquellas con la siguiente forma:
 - $aXb \rightarrow aYb$, donde a y b deben ser terminales, X no terminal e Y cualquiera.
- Gramáticas de tipo 2 → son independientes al contexto. Su forma es:
 - $X \rightarrow Y$, donde X es no terminal e Y puede ser cualquiera.
- Gramáticas de tipo 3 → forma regular. Su forma es:
 - $X \rightarrow Y$, donde X es no terminal e Y puede ser terminal, no terminal o lambda.

Ejemplos

1. $T1 \neq T2$

$X \rightarrow cX \dots \rightarrow bcXa \rightarrow bccXa \rightarrow \dots$

$aXb \rightarrow acXb \dots \rightarrow bcXa \rightarrow$ no se puede seguir derivando X porque para hacerlo se necesita que a la izquierda esté aXb , no solo X como en $T2$.

2. Tipo 3

Handwritten production rules for a Type 3 grammar: $P = \{ A \rightarrow 1B \text{ or } A \rightarrow 0A, B \rightarrow 0 \text{ or } B \rightarrow 0C, C \rightarrow 2 \text{ or } C \rightarrow 2C \}$

3. Tipo 0

Handwritten production rules for a Type 0 grammar: $P = \{ A \rightarrow AZ, A \rightarrow 11B, 1B \rightarrow 01, 1B \rightarrow 0CZ1, C22 \rightarrow 2, C22 \rightarrow 2C \}$

Las gramáticas de tipo 3 son las más precisas, pero las de tipo 2 son las que tienen la mayoría de los lenguajes de programación.